

Relational vs NoSQL

Choose the Right Database for Every Use Case

■■ DATABASES

■ LEARNING OUTCOME

By the end of this module you will understand relational and all major NoSQL database types, apply the CAP theorem to design decisions, design star/snowflake warehouse schemas, and choose the right database for any use case.

■ Skills You Will Gain

- Explain the relational model: tables, keys, normalisation, ACID
- Classify the four NoSQL types: document, key-value, column-family, graph
- Apply the CAP theorem to real database selection decisions
- Design a star schema and a snowflake schema
- Choose between PostgreSQL, MongoDB, Cassandra, Redis, Neo4j
- Understand when denormalisation improves performance

■ Choosing the wrong database is a mistake that costs months to undo. Every senior DE, architect, and backend engineer makes this decision every week.

SECTION 1

Relational Databases — RDBMS

ACID Properties — The 4 Guarantees of Relational Databases

Atomicity: A transaction is ALL-or-NOTHING. If one step fails, the whole transaction rolls back. Consistency: After every transaction, the database remains in a valid state per all defined rules. Isolation: Concurrent transactions don't interfere. Each sees a consistent snapshot. Durability: Once committed, data survives crashes, power failures, and restarts. These guarantees make RDBMS the default choice for financial, medical, and transactional systems.

RDBMS	Best For	Key Features
PostgreSQL	Analytics, general purpose, JSONB support	Open source, extensible, JSONB, window functions, CTE
MySQL	Web applications, LAMP stack	Fast reads, widely hosted, simple to operate
SQL Server	Enterprise Windows environments	Integration with Microsoft ecosystem, SSMS
Oracle DB	Large enterprise, financial systems	RAC clustering, partitioning, mature ecosystem

SECTION 2

NoSQL Database Types

Type	Data Model	Use Case	Examples
Document	JSON / BSON documents	User profiles, product catalogs, CMS	MongoDB, Firestore, CouchDB
Key-Value	Dictionary: key → value	Sessions, caching, leaderboards, queues	Redis, DynamoDB, Memcached
Wide-Column	Rows with dynamic columns	Time-series, IoT, write-heavy workloads	Cassandra, HBase, BigTable
Graph	Nodes and edges	Social networks, fraud, recommendations	Neo4j, Amazon Neptune
Search	Inverted index	Full-text search, log analysis	Elasticsearch, OpenSearch
Time-Series	Timestamps + metrics	Monitoring, IoT, financial ticks	InfluxDB, TimescaleDB
Vector DB	Embedding vectors	Semantic search, RAG for LLMs	Pinecone, Weaviate, pgvector

SECTION 3

CAP Theorem

You Can Choose ONLY 2 of 3 Properties

Consistency: All nodes see the same data at the same time Availability: Every request gets a response (not necessarily the most current data) Partition Tolerance: System continues operating even if network partitions occur CA databases: PostgreSQL, MySQL — no partition tolerance (single node or tight cluster) CP databases: MongoDB, HBase, Zookeeper — consistent but may reject requests during partition AP databases: Cassandra, DynamoDB, CouchDB — available but data may be temporarily stale

Data Warehouse Schema Design

Schema Type	Structure	Query Speed	Storage
Star Schema	Fact table + denormalised dimension tables	Fastest — few joins needed	More redundancy
Snowflake Schema	Fact table + normalised dimensions (sub-dimensions)	Slower — more joins	Less redundancy
One Big Table (OBT)	Everything in one wide flat table	Simplest queries	Very large, hard to update
Data Vault	Hubs (keys) + Links (relationships) + Satellites	Complex queries — very flexible	Designed for historisation

```
-- Star Schema example: fact table with foreign keys to dimension tables
CREATE TABLE fact_orders (
  order_key BIGINT PRIMARY KEY,
  date_key INT REFERENCES dim_date(date_key),
  customer_key INT REFERENCES dim_customer(customer_key),
  product_key INT REFERENCES dim_product(product_key),
  quantity INT,
  unit_price DECIMAL(10,2),
  revenue DECIMAL(10,2),
  cost DECIMAL(10,2),
  profit DECIMAL(10,2) );
-- Dimension table with surrogate key + business key + attributes
CREATE TABLE dim_customer (
  customer_key INT PRIMARY KEY,
  -- surrogate key (auto-increment)
  customer_id VARCHAR(20),
  -- business key from source system name
  name VARCHAR(100),
  city VARCHAR(50),
  segment VARCHAR(20),
  valid_from DATE,
  -- SCD Type 2 history tracking
  valid_to DATE,
  is_current BOOLEAN );
```

Choosing the Right Database

Use Case	Best Choice	Why
E-commerce orders, transactions	PostgreSQL / MySQL	ACID guarantees, relational integrity
Product catalog, user profiles	MongoDB	Flexible schema, nested JSON documents
Session storage, caching	Redis	Sub-millisecond reads, in-memory
Real-time leaderboard	Redis Sorted Sets	O(log n) rank operations
IoT sensor data, metrics	Cassandra / InfluxDB	Write-heavy, time-ordered, scalable
Fraud detection, social graph	Neo4j	Multi-hop relationship traversal
Log search, observability	Elasticsearch	Full-text search, aggregations
Analytics reporting, BI	Snowflake / BigQuery	Columnar, MPP, SQL at petabyte scale
ML feature store / RAG	Pinecone / pgvector	ANN vector similarity search

■ PRACTICE Q & A

- Q1.** What does ACID stand for and why does it matter for financial systems?

A: Atomicity, Consistency, Isolation, Durability. In banking, a debit from one account must be matched by a credit to another — if any step fails, both must roll back. ACID prevents partial updates.
- Q2.** When would you choose Cassandra over PostgreSQL?

A: Cassandra for write-heavy workloads at massive scale (IoT, time-series, global distribution). PostgreSQL for transactional data requiring ACID, complex joins, and relational integrity.
- Q3.** What is a surrogate key and why use it in a data warehouse?

A: An artificially generated integer primary key for dimension tables (e.g. auto-increment). Decouples the warehouse from source system keys, enables SCD Type 2 history, and speeds up joins.
- Q4.** What is the CAP theorem and how does it affect database selection?

A: In a distributed system you can only guarantee 2 of: Consistency, Availability, Partition tolerance. Cassandra chooses AP (available + partition tolerant). PostgreSQL chooses CA (single node, no partition).

Q5. What is the difference between OLTP and OLAP databases?

A: OLTP (Online Transaction Processing): many small read/write transactions, row-oriented, normalised (PostgreSQL, MySQL). OLAP (Online Analytical Processing): complex queries over millions of rows, columnar, denormalised (Snowflake, BigQuery).

■■ Database Selection Cheat Sheet	
PostgreSQL	ACID relational – general purpose
MySQL	Web apps, LAMP stack
MongoDB	Document DB – flexible JSON schema
Redis	Key-value – cache, sessions, queues
Cassandra	Wide-column – write-heavy, distributed, AP
Neo4j	Graph – relationship-heavy data
Elasticsearch	Full-text search, log analysis
Snowflake	Cloud data warehouse – SQL analytics
BigQuery	GCP serverless warehouse – pay per query
Redshift	AWS columnar MPP warehouse
InfluxDB	Time-series – IoT, monitoring
Pinecone	Vector DB – semantic search, RAG